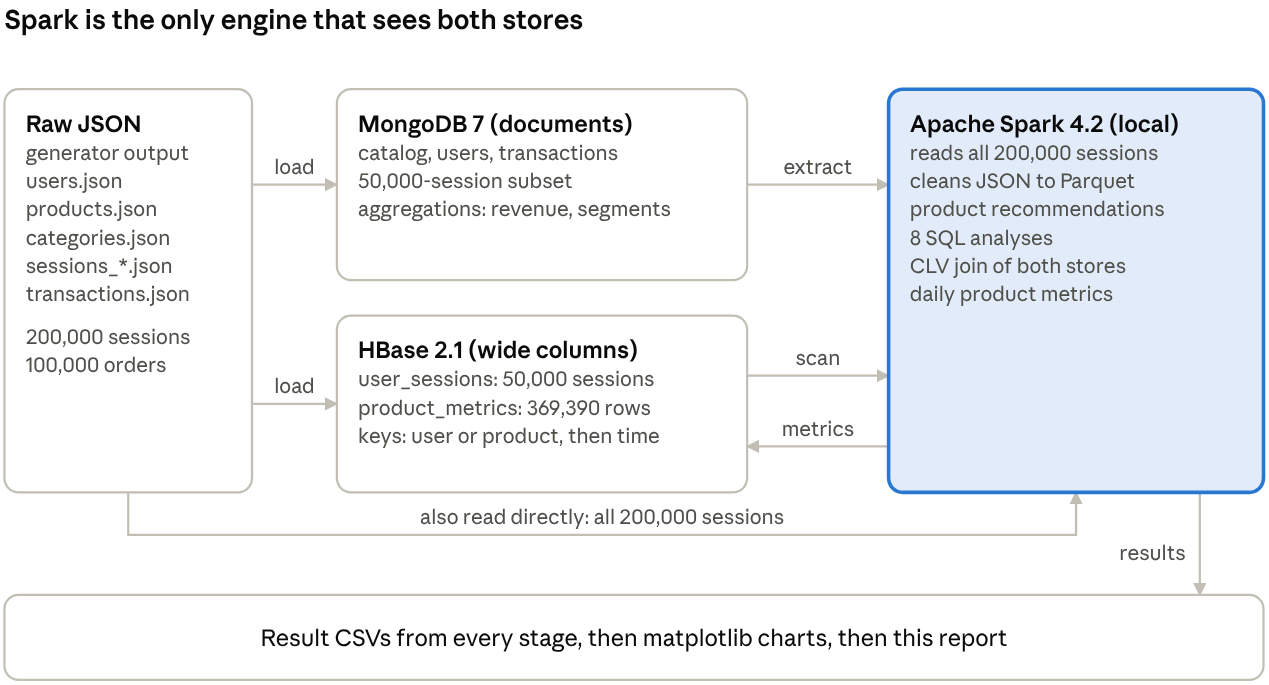
Distributed Multi-Model Analytics for E-commerce Data: Technical Report

AUCA Big Data Analytics Final Project · Nov 10, 2026 · @Yves

Student: NISHYIRIMBERE Yves · **ID:** 101388 · **Course:** Big Data Analytics, AUCA

1. Introduction and system architecture

MongoDB, HBase and Apache Spark each hold the part of a 90-day e-commerce dataset that suits their data model, and Spark joins them for analytics that no single store can answer alone.



system architecture · 3 engines, 1 integration point

Raw JSON is loaded into MongoDB and HBase, while Spark reads the raw files directly, extracts from both stores for the integrated analysis, and writes daily product metrics that are loaded into HBase.

The data is synthetic, produced by the provided generator and scaled to 10,000 users, 5,000 products in 25 categories, 200,000 sessions (about 2.0 million page views) and 100,000 transactions, covering roughly August to November 2026. MongoDB 7 and HBase

2.1.3 run in Docker; Spark 4.2.0 runs in local mode on a MacBook, and HBase is reached through its Thrift server with the happybase client.

Each system has a distinct job:

- **MongoDB** stores the catalog, user profiles, transactions with embedded line items, and a 50,000-session subset with embedded page views. It serves document-shaped analytics such as revenue by category and customer segmentation.
- **HBase** stores 50,000 sessions as a time-ordered activity stream per user, plus 369,390 product-by-day metric rows. It serves fast key and range lookups.
- **Spark** processes all 200,000 sessions, cleans the raw JSON into partitioned Parquet, computes recommendations and SQL analytics, and joins extracts from MongoDB and HBase for the integrated lifetime-value analysis.

Only a subset of sessions is loaded into the two stores (50,000 of 200,000) to keep loading fast, while Spark always sees the full data. The charts in section 5 are produced from the result files each stage writes.

2. Data modeling: MongoDB and HBase

MongoDB holds data that is read as whole documents and aggregated along many dimensions, while HBase holds data that is read by key and time range.

MongoDB schema

Collection	Design	Reason
products	Category name and subcategory (id, name, profit margin) embedded; price_history array embedded	A product page needs one read, and the history is only read with the product
users	Profile plus an embedded purchase_summary (orders, spend, average order value, first and last purchase), written by a $merge pipeline	Profile reads need no scan of the transactions

Collection	Design	Reason
transactions	Line items embedded; each item also carries category_id and subcategory_id copied from products	An order is always read with its items, and revenue by category needs no join
sessions (50,000 subset)	page_views and cart_contents embedded in the session document	One read returns the whole user journey

Indexes follow the query patterns: transactions on user_id, timestamp and items.product_id; products on category_id; sessions on (user_id, start_time) and conversion_status.

Five aggregation pipelines run on this model: (A1) revenue by category and month using $unwind and $group; (A2) customer segmentation by order count using $bucketAuto, which keeps segments balanced at any data size; (A3) top-selling products from transactions against most-viewed products from sessions; (A4) revenue by US state using $lookup into users; and (A5) the $merge pipeline that writes the purchase summary into each user.

HBase schema

Table	Row key	Column families	Query it serves
user_sessions	user_id#reverse_ts#session_id, where reverse_ts = 9999999999999 minus epoch milliseconds	info (one value per session), pv (sparse page-view columns pv_000, pv_001, ...), cart (sparse, qualifier = product_id)	Latest N sessions of a user; all sessions in a time window
product_metrics	product_id#yyyymmdd	m (views, cart_adds, purchases, revenue)	Daily history of one product over a date range

The reverse timestamp stores each user's sessions newest first, so "latest N sessions" is a short prefix scan and a time window is a start/stop-row range scan. Because the key leads with user_id and not with time, writes spread across regions instead of piling onto the newest rows. There are only three column families because each family is stored separately, and queries that need just the summary read the info family and skip the wide pv family.

On the loaded data, 50,000 sessions loaded in 15.3 seconds and 369,390 product-day rows in 11.9 seconds. In single runs, fetching a user's latest five sessions took 22.0 ms, a 30-day window 17.6 ms, and a product's 78-day history 32.1 ms.

Why each dataset sits where it does

Data	Store	Reason
Catalog, users, transactions	MongoDB	Nested documents aggregated by many fields (category, month, user, state)
Session event stream	HBase	High volume, append-only, always fetched by user and time
Daily product metrics	HBase	Fixed-shape counters read by product and date range, written in bulk by Spark
Session subset with full journeys	MongoDB	A complete journey is analysed as one document, for example most-viewed products

3. Spark processing pipelines

Spark turns 200,000 raw sessions and 100,000 transactions into clean, date-partitioned Parquet, then derives product recommendations and eight SQL analyses from it.

Cleaning and normalisation

The cleaning job reads every raw JSON file with explicit schemas and not inferred ones. This matters because cart_contents is keyed by product_id, so inference would create a struct with thousands of fields; it is modelled as a map instead. The job then parses timestamps, removes duplicates on primary keys, fills missing values, standardises text, and flags data-quality problems instead of silently dropping rows. Nested structures become flat fact tables: 1,997,108 page views, 207,679 cart items and 226,936 transaction

items. Products carry no subcategory in the generated data, so one is assigned deterministically from an MD5 hash of the product id, using the same rule in Python (MongoDB) and Spark; the two agree for all 5,000 products in a test run. Output is Parquet, partitioned by date for the large tables.

Data-quality check	Result
Sessions and transactions in vs out	200,000 and 100,000, none lost
Transactions with no session link	59,512 (59.5%)
Transactions whose total or items do not add up	0
Converted sessions with no transaction	0
Sessions: abandoned / browsed / converted	89,436 / 70,076 / 40,488
Inactive products	234 of 5,000
Users last active before registering	0

Product recommendations

The recommendation job counts how often two products appear in the same basket, using a self-join on basket id restricted to product_a < product_b so each pair is counted once. For each pair it computes support, confidence ("customers who bought A also bought B") and lift (support relative to chance). It runs twice: with transactions as baskets ("bought together") and with sessions as baskets ("viewed together"). A minimum pair count removes one-off coincidences.

The job kept 1,487 bought-together pairs (average lift 82.1, maximum 340.1) and 2,139 viewed-together pairs (average lift 26.6, maximum 125.8). These lifts are inflated by small counts and are not real affinities. In the top pair, the products appear in only 49 and 12 of the 100,000 baskets, so two shared baskets alone give a lift of 340. The generator picks products at random, so the method is sound but there is no real signal for it to find.

Spark SQL analyses

Eight queries run over views of the Parquet tables:

Query	Technique
Q1 Top 3 categories per month	CTE and RANK() window
Q2 Conversion by device and by referrer	GROUPING SETS
Q3 Daily revenue with 7-day moving average	Window frame
Q4 Customer spend quartiles and engagement	Three-way join and NTILE
Q5 Revenue vs estimated profit by category	Joins to dimension tables
Q6 Strict conversion funnel	Conditional aggregation
Q7 Product views vs units sold	Two aggregates joined to the catalog
Q8 Buyers by country: spend and engagement	Join of MongoDB users with HBase sessions, each read with its native client

Optimisation choices

Explicit schemas avoid a full pass over the JSON to infer types. Parquet is columnar, so later jobs read only the columns they need, and date partitioning allows time-range pruning. Reused DataFrames are cached, and the number of shuffle partitions is set to 16 to suit a single machine. The raw files are one JSON array each, which Spark cannot split, so reading parallelism equals the number of files; emitting JSON Lines would remove that limit.

4. Integrated analytics: customer lifetime value

Joining MongoDB purchase history with HBase session activity in Spark ranks 9,998 buyers by estimated lifetime value and shows that browsing engagement barely relates to spend.

Business question. Which customers are the most valuable going forward, how concentrated is revenue in them, and does browsing engagement go together with spend?

Data sources.

- MongoDB: each user's profile and embedded purchase summary (orders, spend, average order value, country).
- HBase: per-session duration and outcome from user_sessions, for the 50,000 loaded sessions.
- Spark: the only engine that sees both, so it performs the join and the computation.

Processing steps.

1. Extract users and purchase summaries from MongoDB with pymongo into a Spark DataFrame with an explicit schema.
2. Scan four small columns of user_sessions from HBase with happybase, never the wide page-view family, into Spark.
3. Aggregate sessions per user: count, average duration, converted and abandoned sessions.
4. Left-join on user_id, so users with no sessions in the HBase subset keep a count of 0.
5. Compute CLV, then rank buyers with percent_rank into four value tiers.

```
\text{CLV} = \text{AOV} \times \frac{\text{orders}}{2.96\ \text{months observed}} \times 12\ \text{months expected lifespan}
```

The 90-day window equals 2.96 months, and the 12-month lifespan is an explicit assumption that can be changed with a parameter. The result is meant for ranking customers, not for forecasting revenue.

Tier	Customers	Avg CLV ($)	Avg orders	Avg sessions (HBase)	Share of total CLV
Platinum (top 10%)	1,000	77,520	15.0	5.3	17.2%
Gold (next 20%)	2,000	59,037	12.2	5.2	26.2%
Silver (next 30%)	2,999	46,065	10.3	5.1	30.7%
Bronze (bottom 40%)	3,999	29,156	7.4	4.8	25.9%

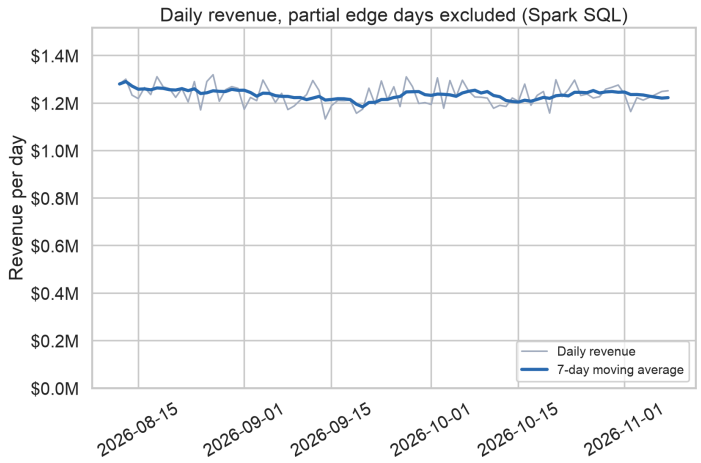
Value is not concentrated: the top 10% of buyers hold only 17.2% of total CLV, while the bottom 40% hold 25.9%. Engagement is nearly unrelated to value: the correlation between session count and total spend is 0.092, and between average session length and CLV is -0.029. The session counts here are small (about 5 per user) because they come from the 50,000-session HBase subset, while the full Spark data averages 19 to 21 sessions per user.

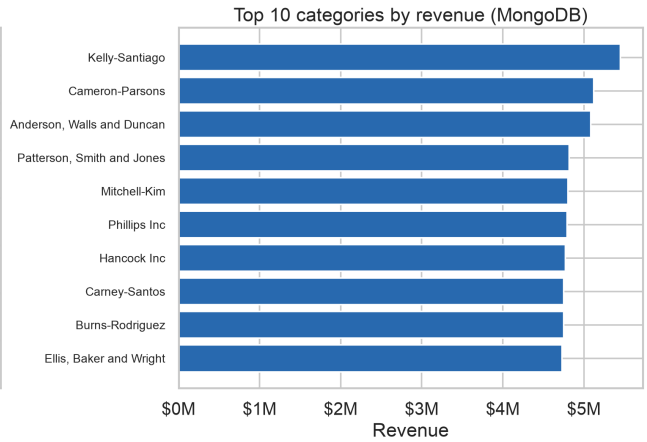
Performance notes. Only four tiny HBase columns are scanned, with a large scan batch, and the MongoDB extract projects only the fields it needs. The join is against a 10,000-row user table. The global percent_rank window runs in a single partition, which is fine at 10,000 buyers; at larger scale the window would be partitioned or replaced by approximate quantiles.

5. Key findings and visualizations

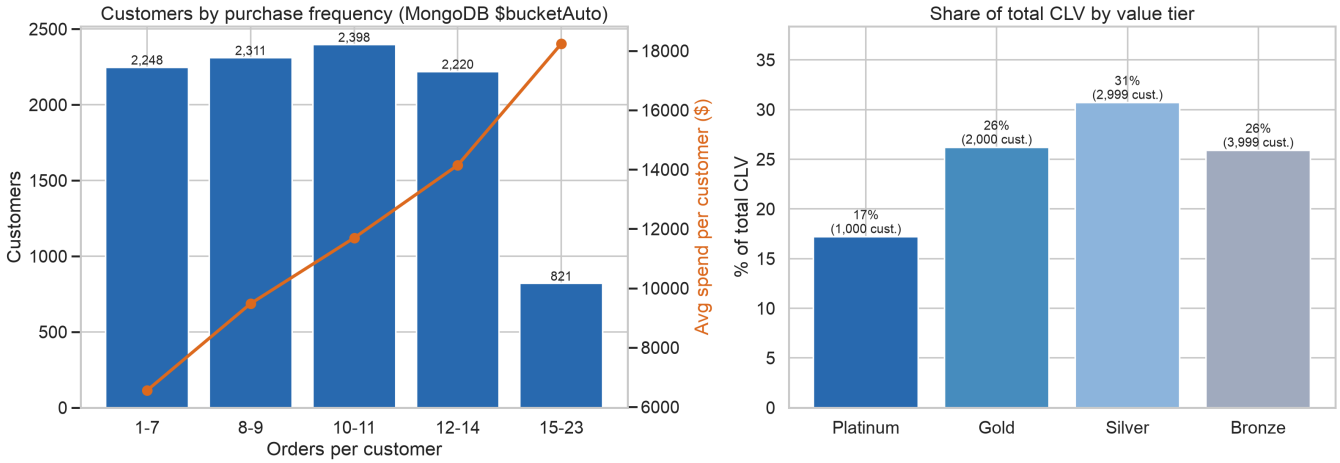
Revenue is steady at about $1.2M a day, customers differ mainly in how often they buy, and conversion, engagement and product popularity show little structure beyond stock availability. Each chart is produced from the result files written by the MongoDB, Spark and integration stages.

Sales performance (Figure 1). Daily revenue holds between about $1.2M and $1.3M with a median of $1.23M, totalling $111.1M over 100,000 orders (average order about $1,111). The 25 categories each earn $4.0M to $5.5M, and the top 10 account for 43.1% of item revenue. Kelly-Santiago leads with $5.45M and ranks first in August, September and October. By estimated profit it ranks only 16th of 25 (20.4% margin), while Rodriguez-Graham earns the most ($1.61M at a 36.2% margin); the overall estimated margin is 25.2%. These margins rest on the assigned subcategories, so they illustrate the method more than they report real profit.

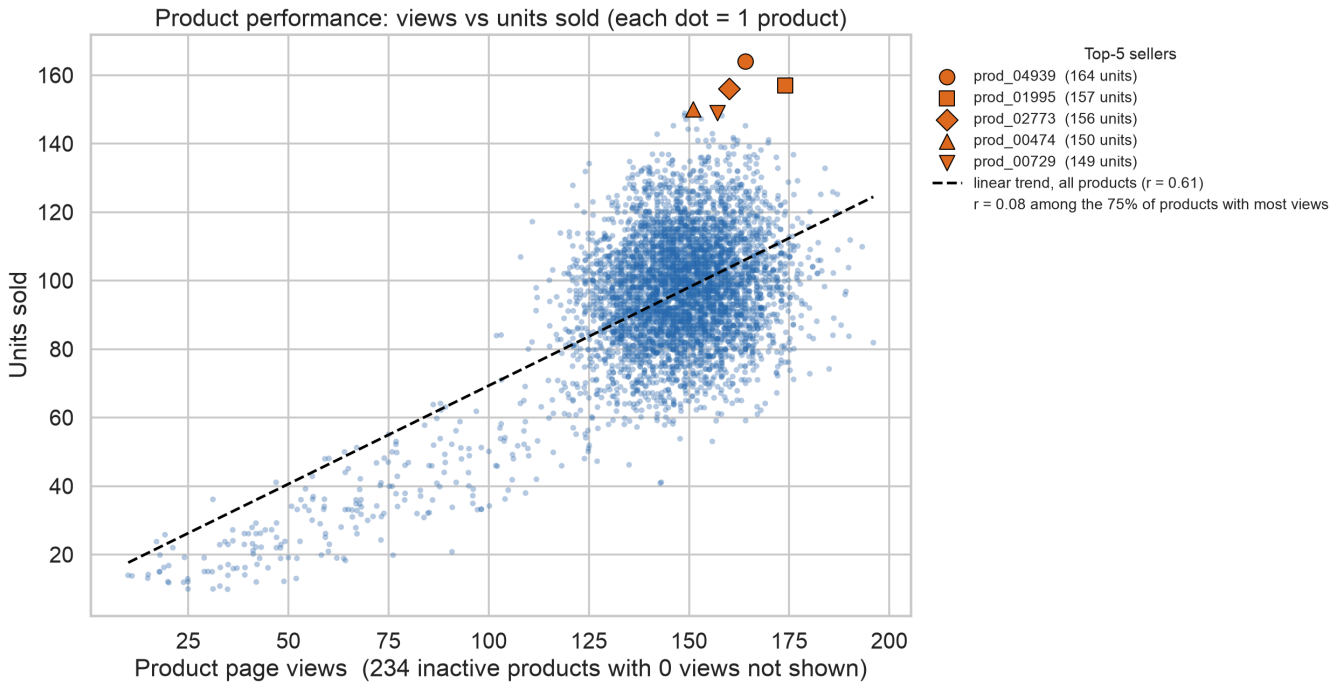




Customer segmentation (Figure 2). Buyers fall into five near-equal groups by order count, and average spend rises from $6,561 (1 to 7 orders) to $18,240 (15 to 23 orders). Revenue is not concentrated in a few customers: the top 10% by CLV hold 17.2% of total CLV. Engagement barely changes with spend: across spend quartiles, average sessions per user move only from 19.0 to 21.1 and session length stays near 1,810 seconds.

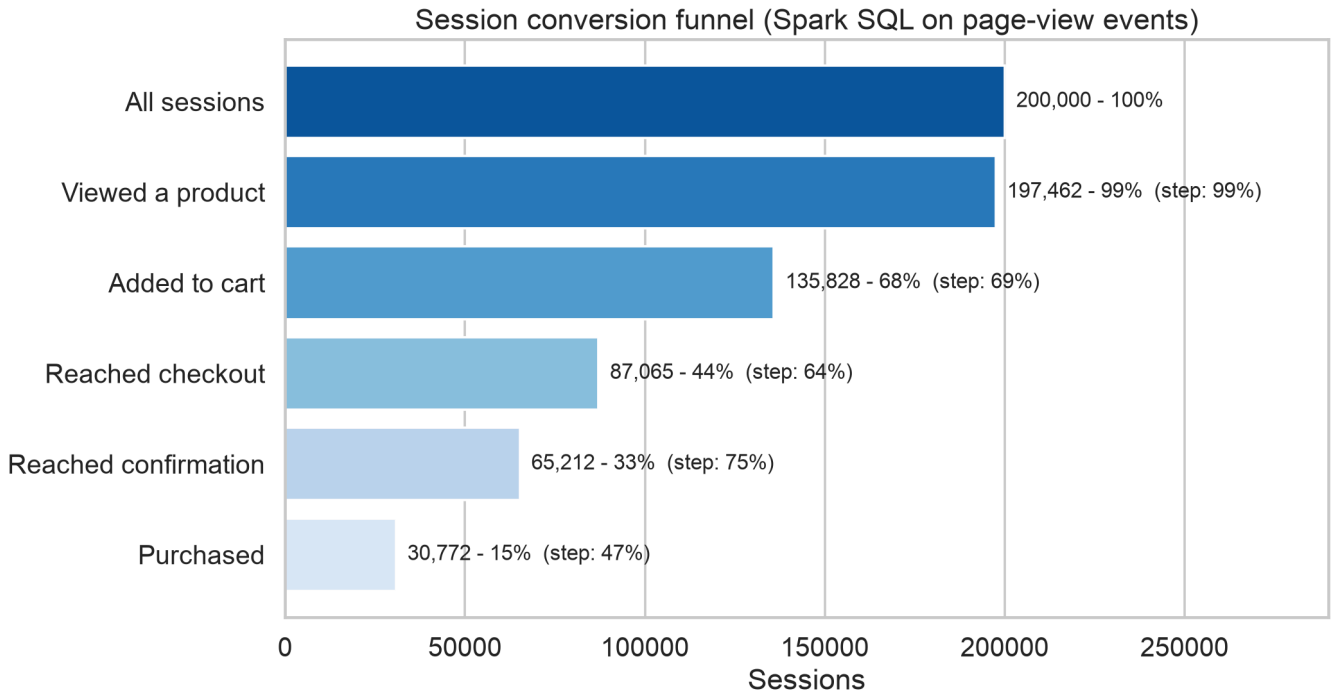


Product performance (Figure 3). Across all active products, views and units sold correlate at r = 0.61, but among the 75% of products with the most views the correlation is only r = 0.08. The overall figure comes from a tail of low-view, low-sales products. In the bottom 10% by views (460 products), 53.9% have zero stock left, against 3.0% of the other products, and their average remaining stock is 204 units against 439. Product age does not explain it: the latest creation date in both groups is June 12, before the data window starts. This is consistent with stock-outs driving the relationship, though it shows association, not proof. The 234 inactive products have no views or sales and are not plotted.

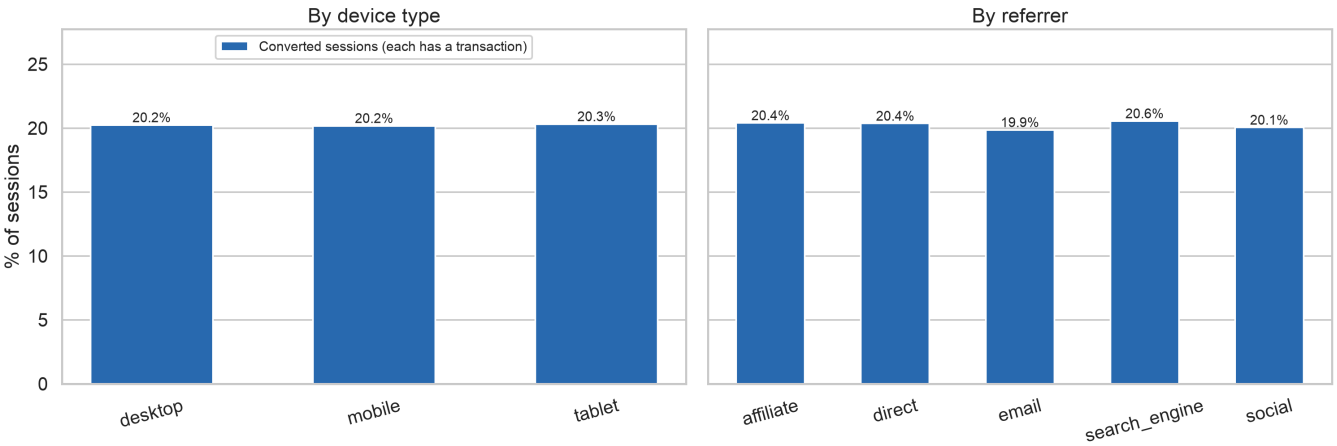


Conversion funnel (Figure 4). 68% of sessions add to cart, 44% reach checkout and 33% reach the confirmation page. Among page-to-page steps, cart to checkout loses the most (64% continue). The strict path ends with 30,772 purchasers (15%), yet 40,488 sessions

(20.2%) are marked converted and each has a transaction, so the page-view path undercounts conversions by about a quarter.



Conversion by device and referrer (Figure 5). Conversion is 20.2% to 20.3% across devices and 19.9% to 20.6% across referrers. No channel stands out, so in this data neither device mix nor traffic source is a lever for conversion.



6. Scalability considerations

Every stage was built so that growing the data means adding machines or partitions, not rewriting the analysis. What ran here is a single-node Docker setup and local-mode Spark on one laptop, so the points below describe how each part would scale and were not measured at larger size.

- **MongoDB.** Transactions and sessions would be sharded on a hashed user_id, which keeps one user's documents together and spreads load evenly. Embedded line items avoid joins across shards, and the denormalised category on each item keeps revenue pipelines to a single collection. The $merge-built purchase summary can be refreshed incrementally.
- **HBase.** Row keys lead with user_id or product_id, so writes spread across regions instead of hot-spotting on the newest timestamp. The tables would be pre-split by key prefix (the shell script includes example splits) and extra region servers added as volume grows. Sparse columns mean empty cells cost nothing.
- **Spark.** The same code runs on a cluster without changes. Parquet with date partitioning lets time-range queries read only the needed files. Small dimension tables such as users and products are joined by broadcast. For the co-occurrence self-join, a minimum count and the product_a < product_b rule cut the pair volume, and very popular products would need skew handling. Ingest should move from one-array JSON files to JSON Lines or straight to Parquet so reading can be split across workers.
- **Integration.** The CLV job pulls extracts through single Python clients, which is fine at 50,000 sessions. At production scale, HBase data would be aggregated inside the cluster (Spark or MapReduce) and the Spark connectors for MongoDB and HBase used in place of client-side extraction.

7. Technology selection by analytical task

Each task uses the technology whose data model matches how the data is read.

Task	Technology	Why this one
Revenue by category and month, segmentation by order count, revenue by state	MongoDB aggregation	Orders arrive as documents with embedded items, and pipelines group them by many fields without joins
One user's latest sessions, or sessions in a time window	HBase	The reverse-timestamp row key makes both a short key-range scan (17.6 to 22.0 ms measured)

Task	Technology	Why this one
Daily metrics for one product	HBase	Keys are product and date, so a range scan returns the whole history
Cleaning and reshaping 200,000 sessions and about 2.0 million page views	Spark	Distributed reads and writes, explicit schemas, columnar Parquet output
Co-occurrence recommendations	Spark	A self-join and aggregation over millions of rows is a distributed job
Funnel, window functions, cross-table SQL	Spark SQL	Declarative queries over Parquet, easy to read and audit
Lifetime value across profiles, transactions and engagement	Spark joining MongoDB and HBase extracts	Neither store holds both inputs, and only Spark can combine them and compute over the result

8. Limitations and future work

The pipeline works end to end, but the synthetic data limits what the analyses can show, and several choices were simplifications.

Limitations

- **Random data.** The generator draws products and behaviour at random, so co-purchase lifts are chance-level, engagement is flat across spend groups, and conversion barely differs by device or referrer (19.9% to 20.6%).
- **Subsets in the stores.** MongoDB and HBase hold 50,000 of 200,000 sessions. Engagement in the CLV analysis and in the cross-store query therefore reflects a quarter of the sessions.
- **Assigned subcategories.** The data has no product subcategory, so profit margins come from a hash-based assignment. Profit by category is illustrative, not a real margin analysis.
- **No demographics.** Users have no age or similar fields, so segments use order frequency, spend and CLV tier.

- **Funnel definition.** The strict page-path funnel counts 30,772 purchasers, while 40,488 sessions are marked converted and each has a transaction. The page-view path undercounts conversions by about a quarter.
- **Simple CLV.** It extrapolates a 90-day window to 12 months with no churn model. Amounts (about $11,000 per buyer in 90 days) are unrealistic and suit ranking only.
- **Sessions and transactions are loosely linked.** 59.5% of transactions have no session. Dates are relative to when the data was generated, and the first and last days are partial.
- **Single-node setup.** Docker HBase and local Spark show the design but not cluster behaviour, and the CLV extraction uses client libraries instead of the Spark connectors.

Future work

- Use the Spark connectors for MongoDB and HBase and a simulated stream (Structured Streaming) for near-real-time funnel metrics.
- Replace co-occurrence with collaborative filtering (ALS) on data that has real preference structure.
- Replace the simple CLV with a probabilistic model that handles churn.
- Track stock levels over time, to test whether stock-outs drive the low-view, low-sales tail of products described in section 5.
- Re-run on larger data in a multi-node cluster to measure scaling.